Introduction à l’architecture MVVM en SwiftUI

420-249-AH - Programmation mobile iOS

1. Qu'est-ce que MVVM ?

MVVM signifie Model, View, ViewModel.

L'objectif est de séparer les responsabilités de l'application :

Composant	Rôle
`Model`	Données
`ViewModel`	Logique et état de l'écran
`View`	Interface utilisateur

2. Structure générale

```
Utilisateur
     ↓
  View
     ↓
ViewModel
     ↓
  Model
     ↓
ViewModel
     ↓
  View
```

La View affiche les données. Le ViewModel contient la logique nécessaire à l'écran. Le Model représente les données de l'application.

3. Model

Le Model représente les données.

Exemple :

```swift
struct Utilisateur {
    let nom: String
    let email: String
}
```

Le Model ne devrait normalement pas contenir le code d'affichage de l'interface.

4. View

La View représente l'interface graphique.

Exemple :

```
struct ProfilView: View {
    var body: some View {
        VStack {
            Text("Profil")
            Button("Modifier") {
            }
        }
    }
}
```

La View doit idéalement rester simple :

- Affichage
- Interaction utilisateur
- Liaison avec le ViewModel

5. ViewModel

Le ViewModel fait le lien entre la View et le Model.

Il peut contenir :

- L'état de l'écran
- La logique
- La validation
- Les opérations sur les données

6. Un ViewModel observable

Avec le système moderne d'observation de Swift, une classe peut utiliser :

```
@Observable
```

SwiftUI peut alors observer les propriétés utilisées par les vues et mettre à jour l'interface lorsqu'elles changent. Apple documente @Observable comme l'approche moderne pour rendre un modèle observable dans SwiftUI.

```
import Observation

@Observable
class LoginViewModel {
    var email = ""
    var password = ""
    var message = ""
}
```

7. Ajouter de la logique au ViewModel

```
@Observable
class LoginViewModel {
    var email = ""
    var password = ""
    var message = ""

    var formulaireValide: Bool {
        !email.isEmpty && !password.isEmpty
    }

    func connecter() {
        if formulaireValide {
            message = "Connexion réussie"
        } else {
            message = "Veuillez remplir tous les champs"
        }
    }
}
```

La validation n'est plus directement placée dans la View.

8. Utiliser le ViewModel dans une View

Une vue peut conserver une instance observable avec @State. C'est une approche documentée par Apple pour un modèle observable créé directement par la vue.

```
struct LoginView: View {
    @State private var viewModel = LoginViewModel()

    var body: some View {
        Text(viewModel.message)
    }
}
```

Lorsque viewModel.message change, l'interface concernée peut être actualisée automatiquement.

9. Modifier les propriétés depuis les contrôles

Pour créer des bindings vers les propriétés d'un objet @Observable, SwiftUI fournit :

```
@Bindable
```

Apple indique que @Bindable permet notamment de connecter des contrôles comme TextField aux propriétés modifiables d'un objet observable.

```swift
struct LoginView: View {
    @State private var viewModel = LoginViewModel()

    var body: some View {
        @Bindable var viewModel = viewModel
        VStack {
            TextField(
                "Courriel",
                text: $viewModel.email
            )
            SecureField(
                "Mot de passe",
                text: $viewModel.password
            )
        }
        .padding()
    }
}
```

10. Exemple complet

```swift
struct LoginView: View {
    @State private var viewModel = LoginViewModel()

    var body: some View {
        @Bindable var viewModel = viewModel
        VStack(spacing: 20) {
            Text("Connexion")
                .font(.title)
            TextField(
                "Courriel",
                text: $viewModel.email
            )
            SecureField(
                "Mot de passe",
                text: $viewModel.password
            )
            Button("Connexion") {
                viewModel.connecter()
            }
            .disabled(!viewModel.formulaireValide)
            Text(viewModel.message)
        }
        .padding()
    }
}
```

Dans cet exemple, le ViewModel regroupe l'état de l'écran (email, mot de passe, message) ainsi que la logique (validation, connexion), pendant que la View se limite à l'affichage et à la liaison avec les contrôles (TextField, SecureField, Button, Text).

Un Model comme Utilisateur entrerait typiquement en jeu une fois la connexion réussie, pour représenter la personne authentifiée — il n'est pas nécessaire pour cet exemple simplifié.

11. Pourquoi utiliser MVVM ?

Sans séparation, on peut facilement obtenir :

```
Button("Connexion") {
    // validation
    // traitement des données
    // appel réseau
    // gestion des erreurs
    // modification de l'interface
}
```

Toute la logique se retrouve dans la View.

Avec MVVM :

```
Button("Connexion") {
    viewModel.connecter()
}
```

La View devient beaucoup plus simple.

12. MVVM et @State

Il ne faut pas remplacer tous les @State par un ViewModel.

Une donnée très locale à l'interface peut rester :

```
@State private var afficherMotDePasse = false
```

Mais une donnée associée à la logique de l'écran peut être placée dans le ViewModel :

```
viewModel.email
viewModel.utilisateur
viewModel.messageErreur
```

13. Ancienne approche SwiftUI

Dans du code SwiftUI existant, on peut aussi rencontrer :

```
ObservableObject
@Published
@StateObject
@ObservedObject
```

Apple fournit aujourd'hui un chemin de migration d'ObservableObject vers le système @Observable.

Exemple ancien :

```
class LoginViewModel: ObservableObject {
    @Published var email = ""
}
```

Pour nos nouveaux exemples, nous utiliserons principalement @Observable.

14. Organisation des fichiers

Une organisation simple :

```
Models
  └── Utilisateur.swift

ViewModels
  └── LoginViewModel.swift

Views
  └── LoginView.swift
```

Cela permet de séparer clairement les responsabilités.

15. À retenir

Composant	Rôle
`Model`	Données
`View`	Interface
`ViewModel`	État + logique de présentation

Avec SwiftUI moderne :

@Observable permet de rendre un ViewModel observable, et @Bindable permet de créer des bindings vers ses propriétés modifiables.

16. Exercice — Compteur MVVM

Créer :

```
CompteurViewModel
```

avec :

```
var compteur = 0
```

et les méthodes :

```
incrementer()
diminuer()
reinitialiser()
```

Créer ensuite :

```
CompteurView
```

qui affiche :

```
Console
Compteur : 5
[ - ]  [ + ]
[ Réinitialiser ]
```

La View ne doit pas modifier directement :

```
compteur += 1
```

Elle doit appeler :

```
viewModel.incrementer()
```

17. Exercice — Message selon le signe

Ajouter dans le ViewModel une propriété calculée :

```
var message: String
```

qui retourne, selon la valeur du compteur :

Valeur du compteur	Message
`Négative`	"Négatif"
`Égale à 0`	"Zéro"
`Positive`	"Positif"